

## **Guía de Actividad 2 – UML ANÁLISIS Y DESARROLLO DE SOFTWARE**

Thomas Isaza Chalarca

Liney Patricia Ricardo

Sara Velásquez

Instructor: Luis Fernando

Base datos SQL(Relacionales)

Institución: CTMA Sena

Antioquia

2025

## 9Guía de Actividad 2 – UML ANÁLISIS Y DESARROLLO DE SOFTWARE

CENTRO DE TECNOLOGÍA DE LA MANUFACTURA AVANZADA  
MEDELLÍN



**Ficha: 3169892**

### **Palabras claves (key words)**

UML, Clase, Objeto, POO, Modelado de Software.

Preguntas de Introducción: •

¿Qué es un diagrama?

Un diagrama es una representación visual que muestra cómo están relacionados y estructurados los elementos de un sistema o proceso.

- ¿Qué tipos de diagramas conoce o ha utilizado?

Diagramas de clases, casos de uso, actividades, de secuencia y de componentes. Diagramas que ayudan a visualizar diferentes vistas del sistema, enfocándose en aspectos específicos del diseño y funcionamiento

- ¿Cómo se estructura un diagrama?

Un diagrama se estructura mediante símbolos gráficos (como rectángulos, flechas, líneas) que representan elementos del sistema. En los diagramas de clases, por ejemplo, cada clase se muestra mediante un rectángulo con compartimentos para nombre, atributos y comportamientos. La estructura también involucra relaciones entre estos elementos, como asociaciones, herencias o dependencias

- ¿Cómo se construye un diagrama?

Se construye identificando los elementos relevantes del sistema (clases, actores, casos de uso) y sus atributos y funciones, posteriormente se representan gráficamente usando los símbolos adecuados y se establecen relaciones entre ellos para reflejar sus interacciones y dependencias

- ¿Cuál es el objetivo de usar diagramas?

El objetivo principal es facilitar la comunicación, comprensión y documentación del sistema entre diferentes actores (analistas, desarrolladores, usuarios). Los diagramas ayudan a visualizar la estructura y el comportamiento del sistema de forma clara y legible, soportando todo el proceso de análisis y diseño

- ¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?

Porque proporcionan una representación clave y comprensible del sistema, permitiendo detectar errores, entender relaciones complejas, y comunicar el proyecto de manera efectiva. Además, ayudan a mantener la coherencia y orden en el proceso de diseño y desarrollo

Investigue:

¿Cuáles son los tipos de diagrama más relevantes en UML? – Describa al menos 4 de ellos.

○ • **Diagrama de Clases:** Muestra la estructura estática del sistema, incluyendo clases, atributos, métodos y relaciones.

○ • **Diagrama de Casos de Uso:** Representa las funcionalidades del sistema desde la perspectiva del usuario, identificando actores y casos de uso.

○ • **Diagrama de Secuencia:** Visualiza la interacción en el tiempo entre objetos del sistema, mostrando el orden de mensajes y llamadas. ○ • **Diagrama de Actividades:** Describe los flujos de trabajo o procesos dentro del sistema, similar a un diagrama de flujo.

¿Cómo representar instancias de las clases en UML?

En UML, las instancias de las clases se representan en diagramas de objetos mediante un rectángulo similar a la clase, pero con un nombre que incluye el nombre de la clase y valores concretos para los atributos

Elabore un ejemplo de una clase. Con sus atributos y características.

¿UML permite incluir notas, ¿cómo, para qué?

Sí, UML permite agregar notas para aclarar o comentar aspectos importantes del diagrama. Se representan con un rectángulo con la esquina doblada, conectado mediante una línea discontinua a los elementos que se desea

comentar. Se usan para explicar decisiones de diseño, detalles adicionales o instrucciones específicas

¿Qué es un método constructor y para qué se utiliza?

Un método constructor es una operación especial que se ejecuta al crear una instancia de una clase. Su función principal es inicializar atributos y preparar el objeto para su uso. Por ejemplo, en programación, un constructor puede asignar valores predeterminados a los atributos del objeto en el momento de su creación

¿Qué tipos de Relaciones pueden especificarse entre clases?

1. **Asociación:** relación simple donde las clases se comunican o trabajan juntas.
2. **Herencia (generalización/especialización):** indica que una clase hereda atributos y métodos de otra.
3. **Agregación:** relación de "todo y parte", donde una clase contiene a otra, pero el ciclo de vida no necesariamente depende.
4. **Composición:** relación fuerte donde la vida del objeto parte depende del todo.
5. **Dependencia:** una clase usa o depende de otra solo durante un tiempo breve.

### Actividad 1: Diagrama de Casos de Uso en UML

Consulte cómo se crea un Diagrama de Casos de Uso en UML y cuáles son los elementos que lo componen, realice un diagrama para describir un proceso que identifique en su contexto (por ejemplo: el proceso para ir de compras al supermercado, un proceso de matrícula, el proceso para hacer un saque en un partido de tenis, etc.).

Como se crea un diagrama de usos:

- Dibujar el sistema: Se representa como un rectángulo grande que delimita el alcance del sistema, dentro del cual se colocan los casos de uso.
- Colocar los actores: Los actores se dibujan fuera del rectángulo del sistema, generalmente como figuras de palo o iconos.
- Establecer las relaciones: Se conectan los actores con los casos de uso mediante líneas (asociaciones). También se pueden mostrar relaciones entre casos de uso, como "include" (<<include>>) y "extend" (<<extend>>), que se representan con líneas discontinuas y flechas.
- Revisar y refinar: Se verifica que el diagrama refleje correctamente las interacciones y funcionalidades clave, ajustando según sea necesario

Evidencias:



## Descripción del Diagrama de Casos de Uso: Sistema de Gestión Académica

### Título del Sistema

Sistema de Gestión Académica

### Actores del Sistema

#### 1. Estudiante

- Actor principal que utiliza el sistema para gestionar su vida académica
- Representado por una figura humana (stick figure) en el lado izquierdo del diagrama

#### 2. Administrador

- Actor secundario que gestiona y administra el sistema académico

- Representado por una figura humana (stick figure) en el lado derecho del diagrama  
**Casos de Uso del Estudiante (8 casos de uso - color azul claro)**

1. **Consultar Cursos**
  - a. Permite al estudiante ver información de los cursos disponibles
2. **Ver Clases Disponibles**
  - a. Muestra las clases específicas programadas para cada curso
3. **Inscribirse en Clase**
  - a. Funcionalidad principal para registrarse en una clase específica
4. **Cancelar Inscripción**
  - a. Permite al estudiante anular su inscripción en una clase
5. **Ver Mis Inscripciones**
  - a. Consulta personal de todas las inscripciones actuales del estudiante
6. **Consultar Horarios**
  - a. Visualización de horarios y duración de las clases
7. **Verificar Costos**
  - a. Consulta de costos de matrícula de los cursos
8. **Consultar Aulas**
  - a. Información sobre las aulas asignadas a las clases

## **Casos de Uso del Administrador (8 casos de uso - color rosado claro)**

1. **Gestionar Cursos**
  - a. Crear, modificar, eliminar y consultar cursos del sistema
2. **Crear Clases**
  - a. Programar nuevas clases asociadas a cursos existentes
3. **Asignar Aulas**
  - a. Vincular aulas específicas a las clases programadas
4. **Gestionar Estudiantes**
  - a. Administrar la información de los estudiantes en el sistema
5. **Ver Inscripciones**
  - a. Monitorear todas las inscripciones del sistema
6. **Gestionar Aulas**
  - a. Administrar información de aulas (capacidad, nombre, disponibilidad)
7. **Configurar Costos**
  - a. Establecer y modificar los costos de matrícula de los cursos
8. **Generar Reportes**
  - a. crear reportes estadísticos y de gestión del sistema

## **Relaciones del Diagrama**

**Líneas de Asociación (continuas):**

- Cada actor está conectado mediante líneas continuas a sus respectivos casos de uso
- Las líneas indican que el actor **participa** en ese caso de uso específico
- Total: 16 líneas de asociación (8 para cada actor)

#### Elementos Visuales:

- **Flechas:** Incluidas en las primeras líneas de cada actor para indicar la dirección de la interacción
- **Colores diferenciados:** Azul para casos de uso del estudiante, rosado para casos del administrador

#### Distribución Espacial

- **Lado izquierdo:** Estudiante y sus casos de uso
- **Lado derecho:** Administrador y sus casos de uso
- **Centro:** Espacio libre para evitar cruce de líneas
- **Diseño simétrico:** Facilita la lectura y comprensión del diagrama

### Actividad 2: Diagrama de Clases en UML

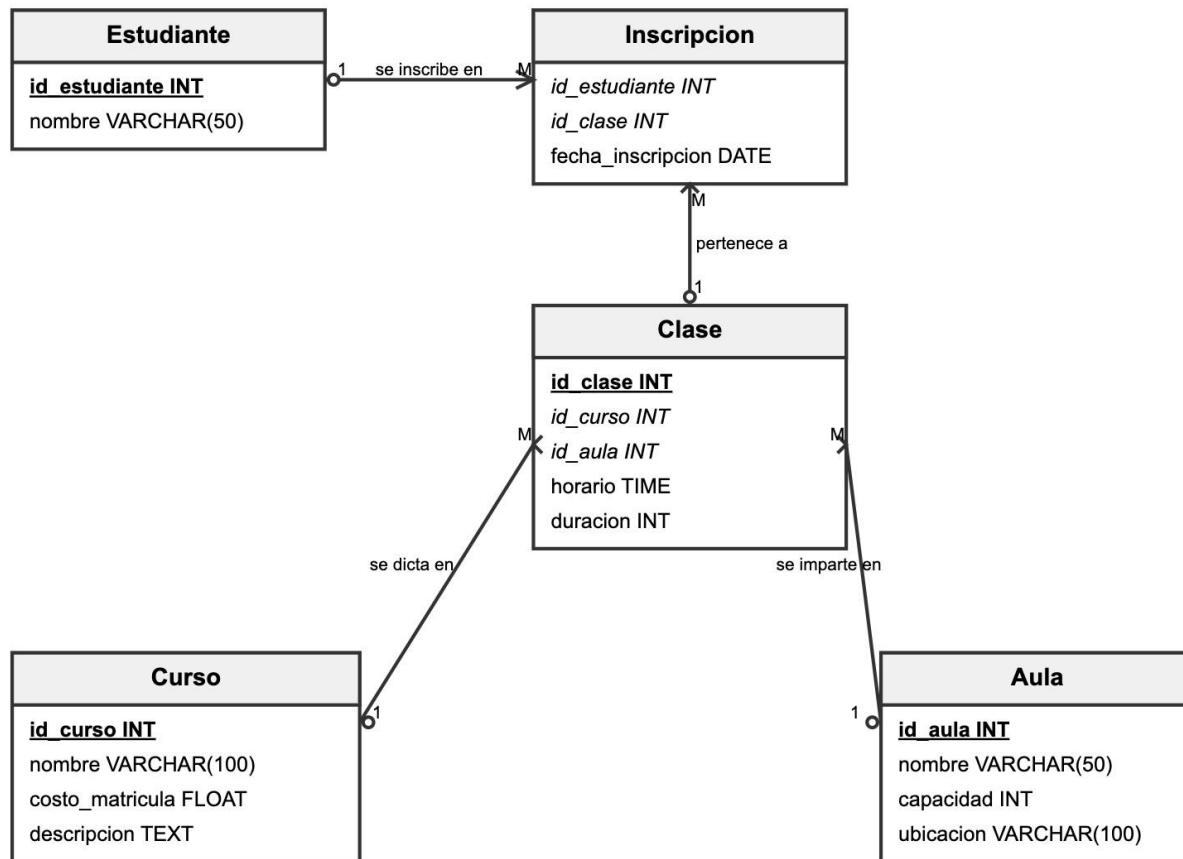
Consulte cuales son los elementos que hacen parte de un diagrama de clases de UML y a partir de la actividad 1, construya una tabla en la cual identifique: verbos, sustantivos, y entidades que intervienen en el caso de uso descrito; con esa información construya el diagrama de clases correspondiente al proceso descrito.

- **Clases:** Son la base del diagrama. Representan entidades o conceptos del sistema y se dibujan como rectángulos divididos en tres partes. La primera sección contiene el nombre de la clase (obligatoria), la segunda lista los atributos (propiedades o características) y la tercera los métodos u operaciones (funciones que la clase puede realizar). Las secciones de atributos y métodos son opcionales según el nivel de detalle deseado.
- **Atributos:** Son las propiedades o datos que describen a una clase, por ejemplo, nombre, edad o precio. Se colocan en la segunda sección del rectángulo de la clase y pueden incluir su visibilidad (público, privado, protegido) usando símbolos como +, -, #.

- Métodos (operaciones): Son las funciones o comportamientos que la clase puede ejecutar, listados en la tercera sección del rectángulo. También pueden tener modificadores de acceso similares a los atributos.
- Relaciones entre clases: Conectan las clases para mostrar cómo interactúan o se relacionan. Los tipos principales son:
  - Asociación: Relación estructural simple, representada con una línea continua.
  - Herencia (generalización): Relación "es un", representada con una línea con flecha triangular abierta hacia la clase padre.
  - Agregación: Relación de "parte-todo" débil, con un rombo vacío en el lado del todo.
  - Composición: Relación de "parte-todo" fuerte y dependiente, con un rombo relleno en el lado del todo
- Interfaces: Definen un conjunto de operaciones que una o varias clases implementan. Se representan como rectángulos con la palabra «interface» o con un círculo conectado a las clases que la implementan.
- Paquetes: Son contenedores que agrupan clases relacionadas para organizar el diagrama. Se dibujan como rectángulos con una pestaña en la esquina superior izquierda.
- Enumeraciones: Tipos de datos definidos por el usuario con un conjunto limitado de valores posibles, también pueden aparecer en el diagrama.
- Objetos: Instancias concretas de una clase, a veces se incluyen para ejemplificar casos específicos.

Evidencias:





## • Descripción de las Clases del Modelo de Gestión Académica

### 1. Clase Estudiante

- **Propósito**
  - Representa a los estudiantes que se inscriben en cursos dentro del sistema académico.
- **Atributos**
  - **id\_estudiante** (INT) - *Clave Primaria*  
Identificador único del estudiante

Generado automáticamente por el sistema

- **nombre** (VARCHAR(50)) Nombre completo del estudiante  
Campo obligatorio para identificación

- **Comportamientos/Métodos**

- **inscribirseEnClase(id\_clase)**: Permite al estudiante inscribirse en una clase específica
- **consultarInscripciones()**: Obtiene todas las inscripciones activas del estudiante
- **consultarHistorialAcademico()**: Muestra el historial completo de clases tomadas
- **cancelarInscripcion(id\_clase)**: Cancela una inscripción existente
- **validarDatosPersonales()**: Valida que los datos del estudiante sean correctos
- 

## 2. Clase Inscripción

- **Propósito**

- Representa la relación entre un estudiante y una clase específica, actuando como tabla de unión con información adicional.

- **Atributos**

- **id\_estudiante** (INT) - *Clave Foránea* Referencia al estudiante inscrito  
Parte de la clave primaria compuesta
- **id\_clase** (INT) - *Clave Foránea*  
Referencia a la clase en la que se inscribe  
Parte de la clave primaria compuesta
- **fecha\_inscripcion** (DATE)  
Fecha en que se realizó la inscripción  
Permite llevar control temporal

- **Comportamientos/Métodos**

- **registrarInscripcion()**: Crea una nueva inscripción validando disponibilidad
- **verificarDisponibilidad()**: Verifica si hay cupo disponible en la clase
- **calcularFechaLimite()**: Calcula fecha límite para cancelación

- **generarComprobante():** Genera comprobante de inscripción
- **validarRequisitos():** Verifica que el estudiante cumpla requisitos
- 

### 3. Clase

- **Propósito**

- Representa una sesión específica de un curso, con horario, aula y duración definidos. Es la entidad central que conecta cursos, aulas y estudiantes.

- **Atributos**

- **id\_clase** (INT) - *Clave Primaria*  
Identificador único de la clase  
Generado automáticamente
- **id\_curso** (INT) - *Clave Foránea*  
Referencia al curso al que pertenece la clase  
Define el contenido académico
- **id\_aula** (INT) - *Clave Foránea* Referencia al aula donde se imparte  
Define la ubicación física
- **horario** (TIME)  
Hora de inicio de la clase  
Formato 24 horas (HH:MM)
- **duracion** (INT)  
Duración de la clase en minutos  
Permite calcular hora de finalización

- **Comportamientos/Métodos**

- **programarClase():** Programa una nueva clase verificando disponibilidad
- **verificarDisponibilidadAula():** Verifica que el aula esté libre en el horario
- **calcularHoraFin():** Calcula hora de finalización basada en duración
- **consultarInscritos():** Obtiene lista de estudiantes inscritos
- **verificarCapacidad():** Verifica si hay cupo disponible
- **reprogramarClase():** Permite cambiar horario o aula
- **cancelarClase():** Cancela la clase y notifica a estudiantes
-

## 4. Clase Curso

- **Propósito**
  - Representa un programa académico completo con contenido, costo y descripción específicos.
- **Atributos**
  - **id\_curso** (INT) - *Clave Primaria*  
Identificador único del curso  
Generado automáticamente
  - **nombre** (VARCHAR(100))  
Nombre descriptivo del curso  
Debe ser único en el sistema
  - **costo\_matricula** (FLOAT)  
Costo de inscripción al curso  
Valor monetario en moneda local
  - **descripcion** (TEXT)  
Descripción detallada del contenido del curso  
Incluye objetivos y prerequisites
- **Comportamientos/Métodos**
  - **crearCurso()**: Crea un nuevo curso en el sistema
  - **actualizarCosto()**: Modifica el costo de matrícula
  - **consultarClasesProgramadas()**: Obtiene todas las clases del curso
  - **calcularTotalEstudiantes()**: Cuenta estudiantes inscritos en todas las clases
  - **generarReporteCurso()**: Genera reporte estadístico del curso
  - **validarPrerequisitos()**: Valida prerequisites para inscripción

## 5. Clase Aula

- **Propósito**
  - Representa un espacio físico donde se imparten las clases, con capacidad y ubicación específicas.

- **Atributos**
- **id\_aula** (INT) - *Clave Primaria*  
Identificador único del aula  
Generado automáticamente
- **nombre** (VARCHAR(50))  
Nombre o código del aula  
Debe ser único (ej: "A-101", "Lab-Sistemas")
- **capacidad** (INT)  
Como Número máximo de estudiantes que puede albergar Valor entero positivo
- **ubicacion** (VARCHAR(100))  
Descripción de la ubicación física  
Incluye edificio, piso, referencias
- **Comportamientos/Métodos**
- **verificarDisponibilidad()**: Verifica si el aula está libre en un horario
- **consultarOcupacion()**: Obtiene horarios ocupados del aula
- **calcularPorcentajeUso()**: Calcula porcentaje de ocupación
- **programarMantenimiento()**: Programa mantenimiento del aula
- **generarReporteUso()**: Genera reporte de uso del aula
- **validarCapacidad()**: Verifica que no se exceda la capacidad
- 

## Relaciones entre Clases

- **Cardinalidades**
- **Estudiante → Inscripcion**: 1:M (Un estudiante puede tener múltiples inscripciones)
- **Clase → Inscripcion**: 1:M (Una clase puede tener múltiples inscripciones)
- **Curso → Clase**: 1:M (Un curso puede tener múltiples clases)
- **Aula → Clase**: 1:M (Un aula puede albergar múltiples clases)
- **Reglas de Negocio**
- Un estudiante no puede inscribirse dos veces en la misma clase

- Una clase no puede exceder la capacidad del aula asignada
- No pueden programarse dos clases en la misma aula al mismo tiempo
- Los horarios de clase deben ser válidos (no negativos, no exceder 24 horas)
- El costo de matrícula debe ser un valor positivo
- **Integridad Referencial**
  - Al eliminar un **Estudiante**, se deben eliminar sus **Inscripciones**
  - Al eliminar una **Clase**, se deben eliminar sus **Inscripciones**
  - Al eliminar un **Curso**, se deben eliminar sus **Clases** e **Inscripciones** • Al eliminar un **Aula**, se deben reasignar o eliminar sus **Clases**

## GLOSARIO

1. **UML (Lenguaje de Modelado Unificado):** Es un lenguaje visual estandarizado que permite especificar, visualizar, construir y documentar sistemas de software mediante diagramas y notaciones gráficas. Facilita la comunicación entre desarrolladores, analistas y stakeholders durante todo el ciclo de vida del sistema.
2. **POO (Programación Orientada a Objetos):** Paradigma de programación que organiza el software en objetos que contienen datos y comportamientos, promoviendo la reutilización, modularidad y escalabilidad del código.
3. **Objeto:** Es una instancia concreta de una clase que representa una entidad del mundo real o conceptual en el sistema, con atributos (propiedades) y métodos (funciones o comportamientos).
4. **Clase:** Es un modelo o plantilla que define la estructura y comportamiento común a un conjunto de objetos, especificando atributos y métodos que estos objetos compartirán.
5. **Modelo:** Es una abstracción simplificada de una realidad que sirve para entender, diseñar o representar un sistema, permitiendo explorar diferentes perspectivas del mismo.
6. **Atributo:** Es una propiedad o característica de una clase u objeto, que almacena información relevante, típicamente con un nombre y un tipo de dato.
7. **Alcance:** Es la visibilidad o accesibilidad de los atributos y métodos de una clase u objeto, pudiendo ser público, protegido o privado, según quién puede acceder a ellos.

8. **Parámetro:** Es una variable que se pasa a un método o función al momento de su invocación, permitiendo que esta reciba valores específicos para su ejecución.
9. **Agregación:** Es una relación "todo-parte" donde una clase contiene referencias a otras clases (componentes), pero la existencia de los componentes puede ser independiente del todo.
10. **Herencia:** Mecanismo por el cual una clase (subclase) hereda atributos y métodos de otra clase (superclase), promoviendo la reutilización y especialización.
11. **Polimorfismo:** Capacidad que tienen los objetos de diferentes clases relacionadas por herencia para responder a una misma operación de distinta forma, dependiendo del objeto que la ejecuta.
12. **Extensión:** Incorporación de nuevos elementos, atributos o funciones a una clase o sistema existente sin alterar su estructura original, a menudo mediante herencia o composición.
13. **Función:** Es un bloque de código que realiza una tarea específica, puede recibir parámetros y retornar un valor, y se utiliza para modularizar y reutilizar comportamientos.
14. **Relación:** En UML, es una conexión entre clases u objetos que indica cómo interactúan o dependen entre sí, como asociación, herencia, agregación o dependencia.
15. **Abstracción:** Proceso de identificar las características esenciales de una entidad o sistema, ocultando detalles irrelevantes, para centrarse en aspectos relevantes para el diseño o análisis.
16. **Sobrecarga:** Permite definir múltiples funciones o métodos con el mismo nombre en una misma clase, diferenciándose por el número o tipos de parámetros que reciben.

#### Evidencias:

Complete las descripciones de los términos indicados en el glosario anterior, utilice como referencia el contexto de UML y desarrollo de software.

Incluya su glosario en el informe de la actividad de aprendizaje.

#### **EVIDENCIAS A PRESENTAR**

### **Evidencias por desempeño**

*Desarrollo completo de las actividades planteadas en el documento.*

### **Evidencias de conocimiento**

*Informe en formato PDF con las respuestas a cada una de las preguntas y retos planteados en el documento.*

### **Evidencias de producto**

*Informe en formato PDF con las evidencias gráficas de los procesos ejecutados para cumplir con las actividades y sus correspondientes observaciones, descripciones y conclusiones.*

*En este apartado debe tratar de ser detallado y claro con los instrumentos de evaluación a utilizar, rubricas aplicables o cualquier evidencia esperada.*



## REFERENTES BIBLIOGRÁFICOS

Booch, G., Rumbaugh, J., Jacobson, I.. (2006) UML El lenguaje unificado de modelado. Guía del usuario. 2 Edición. Addison Wesley.

## Cibergrafía

<http://www.uml.org> sitio oficial de UML. Consultado en Octubre 10 de 2022.

## CONTROL DEL DOCUMENTO

|                   | Nombre                       | Cargo      | Dependencia                | Fecha     |
|-------------------|------------------------------|------------|----------------------------|-----------|
| <b>Autor (es)</b> | Luis Fernando Sánchez Pérez. | Instructor | CTMA/TICS-<br>Electrónica. | 20/4/2025 |

## CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

|                   | Nombre                      | Cargo      | Dependencia                | Fecha    | Razón del Cambio                                            |
|-------------------|-----------------------------|------------|----------------------------|----------|-------------------------------------------------------------|
| <b>Autor (es)</b> | Luis Fernando Sánchez Pérez | Instructor | CTMA/TICS-<br>Electrónica. | 9/5/2025 | Modificación de Actividades de Apropiación del conocimiento |